MLX91803/4/5

LF Bootloader User Manual

MARCH 2021

Scope

This document describes the Melexis LF Bootloader firmware (or just Bootloader) that is included in t of both MLX91804 and MLX91805 Software Libraries.

By combining the Bootloader with an application firmware one can provide a wireless way to upgrade the application in the future.

With some guidelines this document together with LF Bootloader source code can be used to build own Bootloader supporting other communication protocols.

Contents

1. Communication protocol

1.1. General concept

By following the TPMS standards the MLX91803/4/5 device integrates LF and RF wireless communication channels. The LF Bootloader uses the input LF channel to receive data and commands from a host system and then to inform it about own state by sending RF telegrams.

1.2. LF telegrams

The LF receiver of the MLX91803/4/5 device supports the On-off keying (OOK) modulation of a low frequency signal in a range from 115 kHz till 140 kHz (typically 125 kHz) with a data rate within a range from 3.6 till 4.4 kbps (typical 3.9 kbps).

The Bootloader is configured to accept LF data encoded by the inverted Manchester where a space '0' corresponds to a transition from "one" to "zero", while a mark '1' corresponds to a transition from "zero" to "one".

Any LF telegram sent to Bootloader has to contain preamble, 9-bit synchronization word (optionally), 16-bit Header (Wake-up ID) with a fixed value 0xCEFA following with LSB first and a payload data:

<table>
<tr><td rowspan="3">Preamble:
LF carrier frequency
for about 8 ms</td><td rowspan="3">Optional Sync word
(see Figure 2)</td><td>1 byte</td><td>1 byte</td><td rowspan="2">Payload data coded in Manchester</td></tr>
<tr><td colspan="2">Header</td></tr>
<tr><td>Fixed value:
0xFA</td><td>Fixed value:
0xCE</td><td>Payload data length can vary:
• from 9 till 64 bytes for data telegrams
• from 2 till 8 bytes for command telegrams</td></tr>
</table>

Figure 1. LF message frame format, MSB follows first.

The preamble is an unmodulated LF carrier signal with duration about 8 ms.

The Synchronization word can be assigned optionally to simplify recognition of the next header and payload data. By default the Bootloader doesn't use the synchronization word. Nevertheless, the Bootloader can be optionally reconfigured to support a synchronization word normally used in TPMS:

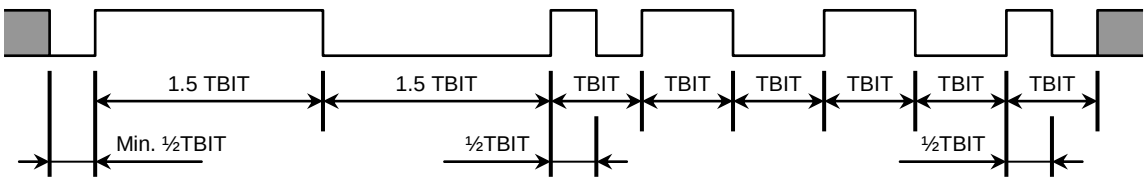

Figure 2. Optional synchronization word of the LF command (TBIT = 256±20 µs).

The LF telegrams deliver either commands or data.

Four available LF telegrams with commands are described in the table 1 below. The field Command ID in the first byte of payload identifies the command type. The commands with the Command ID = 0xE3 and 0xE4 has a fixed payload size = 8. Other two (Set BASIC state and Start Flash Erase commands) can have any payload size from 2 to 8, but with a condition that the lowest byte of the sum of all payload data equals to zero. The shortest acceptable versions of those commands include 2 bytes: 0xE1, 0x1F for the command Set BASIC state, and 0xE2, 0x1E for the command Start Flash Erase.

Table 1: LF commands of the Bootloader

LF command description	Byte_0 Command ID	Byte_1	Byte_2	Byte_3	Byte_4	Byte_5	Byte_6	Byte_7
LFCMD_SET_BASIC **Set BASIC state** **(is not processed in WRITING state)**	0xE1	This field can include any values or be truncated with only condition to have zero at the lowest byte of the sum of all payload data. The shortest command is ended by the **Byte_1 with value 0x1F**.						
LFCMD_ERASE **Start Flash Erase operation** **(processed in ERASING state only)**	0xE2	This field can include any values or be truncated with only condition to have zero at the lowest byte of the sum of all payload data. The shortest command is ended by the **Byte_1 with value 0x1E.**						
LFCMD_SET_ERASING **Set ERASING state** **(processed in BASIC states only)** ADL and ADH are low and high bytes of an address inside of the first Flash sector to be erased. The NUM specifies a decremented by one number of Flash sectors to be erased.	0xE3	0xA5	0x48	0x56	ADL	ADH	NUM	CS
LFCMD_SET_WRITING **Set WRITING state** **(processed in BASIC states only)** CRCL and CRCH are low and high bytes of the 16-bit CRC-16-CCITT (see section 1.4 for details) calculated for 64-byte data to be transmitted and written in the set WRITING state. NUM is a number of the Flash data page. The address of the first byte of any page can be calculated as: Address = 0x6000 + 0x40 ∗ NUM Since anyway the first Flash pages are allocated for the bootloader and hence cannot address application FW, zero NUM addresses the nvRAM.	0xE4	0xA5	0x48	0x56	CRCL	CRCH	NUM	CS

There is an example below how to define a Checksum in the last byte of the payload data:

- *SUM = Byte_0 + Byte_2 + other bytes if defined*
- *Last byte of payload = CS = lowest byte of inverted SUM + 1*

The LF Bootloader disregards the commands with a wrong checksum.

LF telegrams with data are to fill the 64-byte Bootloader Program Buffer needed to program either Flash page, or customer part of nvRAM. Thus, the data telegrams are processed in the WRITING state only and can include any payload data within payload size from 9 till 64 bytes. It is recommended to use the maximal 64-byte payload size to deliver the data, because in this case the Program Buffer is filled by a single LF telegram. Otherwise more than just one LF telegram is needed to fill the Program Buffer.

As soon as the Program Buffer is filled, the Bootloader firmware verifies a CRC of the received data. If CRC is correct, the data is used to write Flash page or nvRAM.

The example of the CRC calculation can be found in the Section 1.4.

1.3. RF telegrams

All received LF commands are confirmed by RF telegrams. That is true even for all detected but then rejected LF commands because of their incorrect Checksum. The RF telegram is also sent after any write operation following the data loading.

If no LF telegram is received for 2 seconds, the Bootloader goes to its BASIC state and starts sending RF telegrams every 10 seconds. If no LF telegram is detected for a one minute, the SW reset is generated in order to try passing the execution to the application.

By default the Bootloader is configured to use 433.92 MHz carried frequency modulated by FSK with a frequency deviation about 40 kHz. The default modulation rate 19200 baud ensures 19200 bps rate for NRZ coded data and 9600 bps rate for the payload data coded in Manchester.

All RF telegrams has similar format shown below:

Preamble: 14 bytes 0xAA coded in NRZ	Sync word: 16-bit word 0x2C34 coded in NRZ	Payload length: 1 byte 0x0A coded in Manchester	1 byte	1 byte	32-bit word	1 byte	1 byte	1 byte	1 byte	16-bit CRC coded in Manchester See example of CRC calculation in the Section 1.4	Termination "0011" coded in NRZ
			Payload data coded in Manchester								
			Header: 0x0C	Bootloader version: 0x01	Chip ID, LSB first (these 4 bytes can be read from nvRAM at address 0x1066...0x1069)	Bootloader State: see Table 2	Counter of failed LF commands	Counter of received LF commands	Number of RF telegram		

Figure 3. Structure of RF telegram

1.4. Example of CRC calculation

The function below calculates the CRC-16-CCITT that is used by Bootloader to ensure integrity of 64-byte Program Buffer and the RFTx payload:

```
uint16_t CalcCRC16_Wmem(const uint16_t StAddr, uint16_t len)
{
    uint8_t i;
    uint16_t j, C;
    uint8_t data;
    uint16_t *Wdata = (uint16_t * const)(StAddr);
    uint16_t crc = 0x0000;                  /* Initialisation value */

    for (j = 0; j < len; j++) {
        if (0 == (j & 0x0001)) {            /* First half of word*/
            data = (uint8_t)((*Wdata)>>8);  /* High Byte */
        }
        else {                              /* second */
            data = (uint8_t)((*Wdata)>>0);  /* Low  Byte */
            Wdata++;
        }
        C = ((crc >> 8) ^ data) << 8;
        for (i = 0; i < 8; i++) {
            if (C & 0x8000) {               /* Test high order bit of crc */
                C = (C << 1) ^ 0x1021;      /* if set, shift & XOR with 0x1021 */
            }
            else {
                C = C << 1;                 /* Else, just shift to left once */
            }
        }
        crc = C ^ (crc << 8);
    }
    return (crc);
}
```

Alternative approach to calculate the CRC-16-CCITT can be found e.g. here:

https://github.com/postmodern/digest-crc/blob/master/lib/digest/crc16_xmodem.rb

2. LF Bootloader State Diagram

2.1. Bootstrap processing

If application firmware is built with the Bootloader, the SW library incorporates an additional logic into the bootstrap procedure to choose between application firmware and the Bootloader for both wake-up events and resets. On the Figure 4 the modified bootstrap logic is marked by orange.

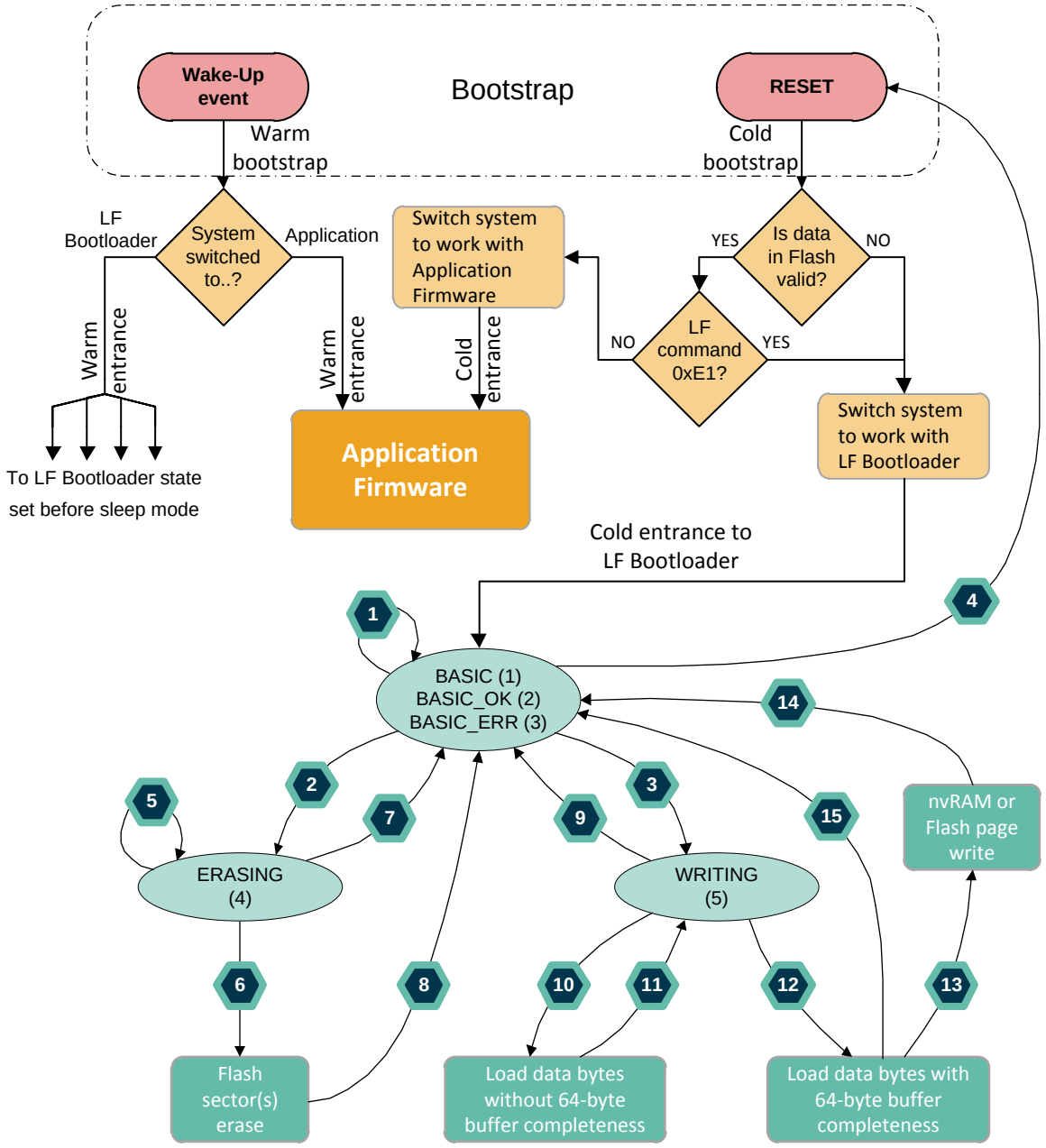

Figure 4. LF Bootloader State Diagram

Note: all transition numbers in the section 2 correspond to the Figure 4 above.

Table 2: List of Bootloader states

State name	State ID	State description
BASIC	0x01	Waiting for LF commands from host system
BASIC_OK	0x02	Basic state after successful operation
BASIC_ERR	0x03	Basic state after failed operation
ERASING	0x04	Waiting for a command LFCMD_ERASE that erases Flash sector(s)
WRITING	0x05	Waiting for LF telegram(s) filling the Program Buffer of the LF Bootloader with the next Flash page or nvRAM write operation

2.2. Basic states

There are three basic states: BASIC, BASIC_OK and BASIC_ERR. All three behave similar but have own unique IDs (please, see Table 2 and Figure 4 for details).

After a reset and the next cold bootstrap, the LF Bootloader starts from the BASIC state. The BASIC state is also set from all conditions, when no LF command is detected for 2 seconds. In this case, being in the BASIC state, the LF Bootloader generates RF telegrams every 10 seconds. Then if no LF telegram is detected for one minute, the SW reset is generated in order to try passing the execution to the application (transition 4).

If erase or write operation is finished successfully the Bootloader goes to the state BASIC_OK. Otherwise, the BASIC_ERR state is set to inform the host system about a failed operation.

All three basic states process LF commands LFCMD_SET_ERASING (transition 2) and LFCMD_SET_WRITING (transition 3). If needed, the command LFCMD_SET_BASIC (transition 1) can be used to go to the pure BASIC state from the states BASIC_OK and BASIC_ERR.

All other kinds of the detected LF telegrams don't change any basic state (transition 1) but provoke sending RF telegram.

2.3. ERASING state

The MLX91803/4/5 Flash memory can be erased by sectors. There are 16 sectors per 1K available.

ERASING is only a single state accepting the Flash sector(s) erase LF command (LFCMD_ERASE). So, in order to erase one or more Flash sectors the host system first has to force the Bootloader to come to the ERASING state by sending the command LFCMD_SET_ERASING (transition 2), and only then the erase operation can be initialized by the command LFCMD_ERASE (transition 6).

The erase operation duration proportionally depends on the number of the Flash sectors assigned for erasing (see NUM field of the LFCMD_SET_ERASING command, Table 1). One sector erase takes about 200 ms. After a successful erase, the Bootloader returns to the state BASIC_OK, otherwise it goes to the state BASIC_ERR (transition 8).

The command LFCMD_SET_BASIC can be used to come back from the ERASING to the BASIC state without erasing (transition 7).

All other LF commands being detected will not change the ERASING state (transition 5).

The transitions 2, 5, 7 and 8 are confirmed by sending RF telegram.

2.4. WRITING state

The WRITING state is used to program the Flash memory or the customer's part of the nvRAM.

The MLX91803/4/5 Flash memory can be programmed by pages only. There are 256 sectors per 64 bytes available. The customer size of the MLX91803/4/5 nvRAM is 64 bytes that equals to the page size of the Flash memory.

In order to write Flash memory or nvRAM, the host system first should send the command LFCMD_SET_WRITING to force the Bootloader to come to the WRITING state (transition 3). Only in the WRITING state the Bootloader accepts the LF telegrams with data needed to fill the Program Buffer assigned in the RAM of the MLX91803/4/5 and then to start the write operation.

Any write operation requires 64 data bytes that have to be delivered in advance to the Program Buffer. 64 data bytes can be delivered either by a single or by multiply LF telegrams. The LF data size is limited by a range from 9 till 64 bytes. Any LF data telegram with payload size < 9 will be rejected with the next transition to the BASIC_ERR state (transition 9).

To get a minimal programming time it is recommended to use the longest 64-byte LF data size to fill the Program Buffer by a single LF telegram (transition 12). If the host system doesn't support so long payload size, the full 64-byte data can be divided by shorter portions to be then sent in its order. Such first LF telegrams follow transitions 10 and 11 without a confirmation via the RF channel. The last portion of the LF data completes filling of the Program Buffer (transition 12). If the CRC assigned in advance in the command LFCMD_SET_WRITING (see Table 1) matches the CRC calculated for the data in the Program Buffer, the write operation is started (transition 13). Otherwise, the data is invalid, the write operation is cancelled, and the system goes to the BASIC_ERR state (transition 15).

The duration of the Flash page write operation is about 50 ms and about 10 ms for nvRAM write operation that is shorter than the data delivery time.

After a successful write, the Bootloader returns to the state BASIC_OK, otherwise it goes to the state BASIC_ ERR (transition 14).

The transitions 3, 9, 14 and 15 are confirmed by sending RF telegram.

3. Process flow example

This section gives an example of the process flow that can be implemented in a customer's host system in order to support the main functionality provided by the LF Bootloader.

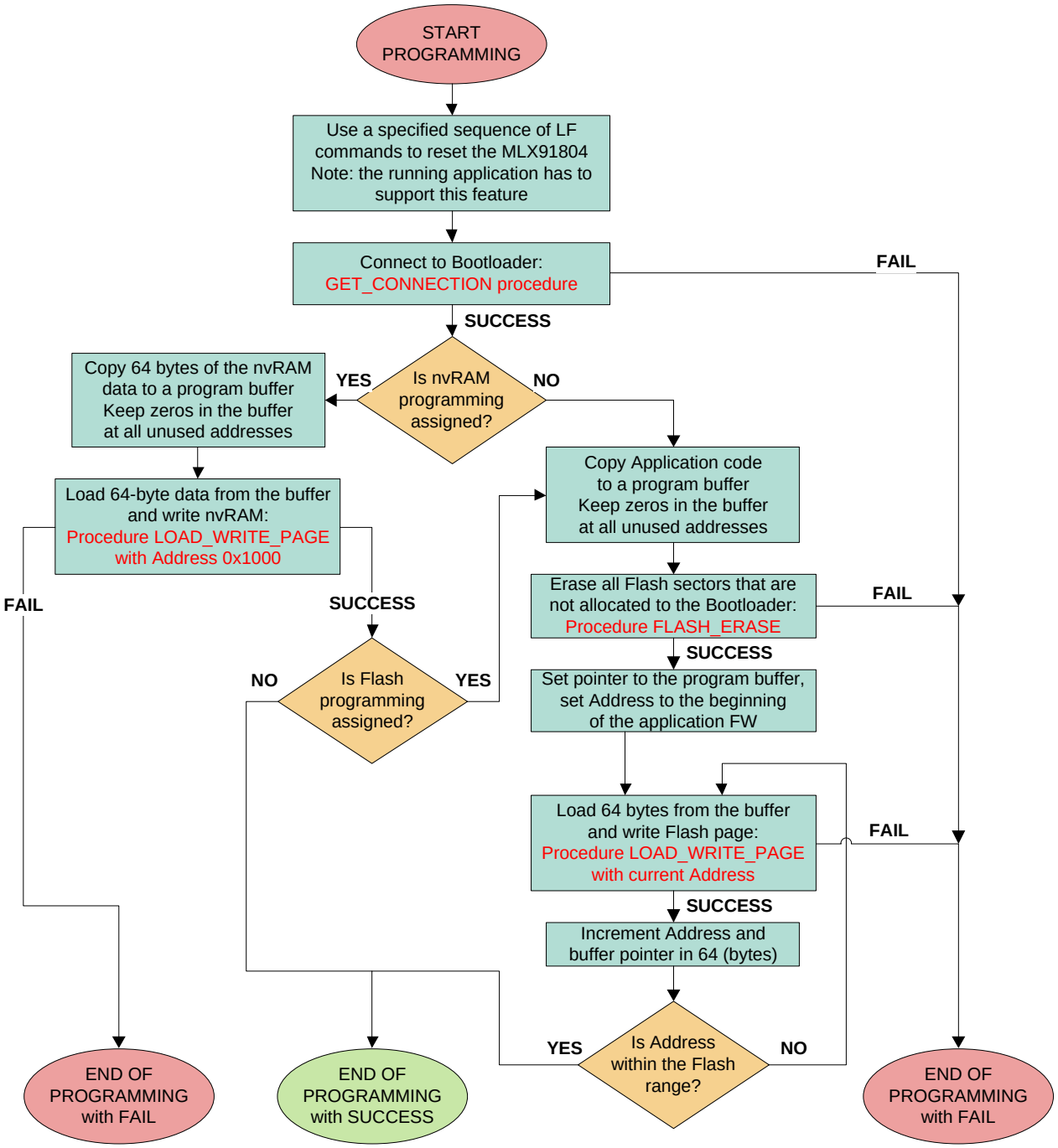

Figure 5. Process flow diagram

Note: After a failed programming, the host system can repeat the process flow some times and only then generate a failure report.

3.1. Address map and HEX files processing

The User Interface (UI) of the host system can request to assign two hex-files with a data needed to program the nvRAM and a data to be used to program application Firmware into the Flash. There are three possible actions available:

- Write nvRAM only
- Write Flash only
- Write Flash and nvRAM

There is a possible scenario of the HEX-files processing below.

The UI software has to prepare two data buffers: one for the nvRAM and another for the Flash memory. Both should be filled by zeros before any next processing.

Since the customer size of the nvRAM includes 64 bytes, a size of the nvRAM data buffer = 64 bytes.

The address range of the nvRAM = 0x1000 ... 0x103F.

The size of the Flash data buffer and the Flash programming range depends on the LOADERSIZE that is a number of Flash sectors occupied by the LF Bootloader. For the SW library the LOADERSIZE is fixed to 3.

Since all MLX91803/4/5 parts have the Flash memory size 16 KB and the sector size 1KB (1024 bytes), one can use two equations below:

The Flash programming range = (0x6000 + LOADERSIZE ∗ 1024) ... 0x9FFF

The size of the Flash data buffer = (16 – LOADERSIZE) ∗ 1024

During processing the UI has to extract from the HEX-file all data that matches the corresponding address range above and put it to the proper place of the buffer above zeros prepared in advance. All data from the HEX-file that doesn't match its address range should be simply disregarded.

3.2. SET_STATE procedure

The SET_STATE procedure is used to force Bootloader to move from any BASIC state towards others and back. The procedure can have two input parameters:

- Desired LF Bootloader state: BASIC, ERASING or WRITING (see table 2).
- Maximal time assigned to achieve the desired state (maximal number of sent LF commands can be used instead).

For example, a denotation SET_STATE (BASIC, 10s) presumes a call of the SET_STATE procedure that is to try setting the BASIC state for 10 seconds.

The procedure SET_STATE has to transmit a sequence of LF commands setting the desired state:

Table 3: LF commands to set the Bootloader states

Desired state name	LF command payload to set the state	Comments
BASIC	0xE1, 0x1F	
ERASING to erase all application area	0xE3, 0xA5, 0x48, 0x56, 0x00, 0x6C, 0x0C, 0x62	For 3K size of the Bootloader: LOADERSIZE = 3
	0xE3, 0xA5, 0x48, 0x56, 0x00, 0x70, 0x0B, 0x5F	For 4K size of the Bootloader: LOADERSIZE = 4
WRITING	0xE4, 0xA5, 0x48, 0x56, CRCL, CRCH, NUM, CS	For Flash: NUM = INT ((Address - 0x6000)/64); For nvRAM: NUM = 0; CRC16 is calculated for the 64-byte data to be programmed; CS is a checksum of LF payload.

Although the short delays between the sending LF commands simplify the commands detection, the delays should not be less than 25 ms. In parallel RF telegrams has to be listened.

The procedure is completed successfully if the desired state is read from any detected RF telegram (see Figure 3). In this case LF commands transmission should be stopped, and the SET_STATE procedure finished.

The SET_STATE procedure fails if after the assigned time or attempts no RF telegram is detected or if the desired state is not found in the detected telegrams.

3.3. Connection to Bootloader (GET_CONNECTION procedure)

The procedure GET_CONNECTION is to force the Bootloader firmware executing. This function can be implemented by the procedure SET_STATE (BASIC, 10s).

If the part is at the cold bootstrap stage happened after any reset, the SET_STATE (BASIC, 10s) procedure switches the execution to the LF Bootloader during the Flash validity (CRC) check (see Figure 4).

If the part already executes the Bootloader firmware, the SET_STATE (BASIC, 10s) procedure will just force the Bootloader to its BASIC state.

3.4. Flash Erase operation (FLASH_ERASE procedure)

FLASH_ERASE procedure is to erase one or more Flash sectors. The procedure has 3 input parameters:

- Address of the first Flash sector to be erased (physical address of any byte inside of the sector).
- Number of sectors to be erased.
- Maximal time assigned to achieve the result (a maximal number of sent LFCMD_ERASE commands can be used instead).

The FLASH_ERASE procedure can be ended either successfully or be failed.

There is a suggested sequence of the FLASH_ERASE procedure:

1) Implement SET_STATE (ERASING, 1s) with the LFCMD_SET_ERASING command having the proper address of the first Flash sector and the number of sectors to be erased (see Table 1 for details).

   It's recommended to erase all sectors at a single FLASH_ERASE procedure before Flash writing. The Table 3 specifies LF command to set ERASING state needed to erase all Flash sectors outside of the Bootloader area for the LOADERSIZE = 3 (default) and 4.

   Finish the FLASH_ERASE procedure with fail if the SET_STATE (ERASING, 1s) procedure fails. Otherwise, go to the next stage below.

2) Transmit a sequence of LFCMD_ERASE commands by keeping a minimal acceptable delay in-between to simplify their detection. Note that the delay should not be less than 25 ms.

   There is a recommended payload of the LFCMD_ERASE command: 0xE2, 0x1E

   In parallel RF telegrams has to be listened.

3) The procedure is completed successfully if any RF telegram with state BASIC_OK (0x02) is detected at the stage 2 above. In this case the LF commands transmission should be stopped, and the FLASH_ERASE procedure finished.

4) The procedure fails if after the assigned time or attempts to send the commands LFCMD_ERASE no RF telegram is detected or if the state BASIC_OK is not found in the detected telegrams.

Note that one sector erase can take up to 0.3 second. This time is multiplied by number of sectors to be erased. It defines a delay till Bootloader goes to the BASIC_OK state with reporting by the RF telegram. In case of fail the Bootloader can go to the BASIC_ERR state earlier, and hence the corresponding RF telegram will be sent as well.

3.5. Write operation (LOAD_WRITE_PAGE procedure)

LOAD_WRITE_PAGE procedure is to write a single 64-byte page of the Flash memory or 64-byte customers' part of the nvRAM. The procedure has 3 input parameters:

- Address of the page to be written. It should be a physical address of any byte inside of the Flash page or a physical address of the first byte of the customer area of the nvRAM (0x1000).
- Pointer to the data inside of the proper buffer loaded in advance from the hex-file (see Section 3.1).
- Maximal number of attempts to achieve the result.

The LOAD_WRITE_PAGE procedure can be ended either successfully or be failed.

There is a suggested sequence of the LOAD_WRITE_PAGE procedure:

1) By using the input data pointer, extract 64 bytes from the proper buffer of the host system.
2) If input address is not 0x1000 (that means a Flash page write operation), verify content of the extracted data. If the data includes all zeros, end the LOAD_WRITE_PAGE procedure with success.
3) Use the C-function example from the section 1.4 to calculate 16-bit CRC for the extracted 64 data bytes. Split the resultant 16-bit word by 2 bytes: CRC_low (CRCL) and CRC_high (CRCH).
4) Based on the input address, calculate the NUM value as it's defined in the Table 3.
5) Implement SET_STATE (WRITING, 1s) with the LFCMD_SET_WRITING command having the CRCL, CRCH and NUM calculated above (see Table 1 for details).

   Finish the LOAD_WRITE_PAGE procedure with fail if the SET_STATE (WRITING, 1s) procedure fails. Otherwise, go to the next stage below.
6) Clean a counter of attempts to write.
7) Clean buffer of received RF telegrams and start listening for new RF telegrams.
8) Send a telegram with a payload including 64 data bytes following according to its address from lowest to highest: Data[00], Data[01], Data[02], […], Data[62], Data[63]
9) The procedure is completed successfully if after transmission above any RF telegram with state BASIC_OK (0x02) is detected. In this case the LF commands transmission should be stopped, and the LOAD_WRITE_PAGE procedure finished.
10) Otherwise, if the RF telegram with state BASIC_ERR (0x03) or just state BASIC (0x01) is detected, go to the stage 11.

    If in 50 ms after the data transmission no RF telegram is detected go to the stage 12.

    If detected RF telegram includes any unexpected state (e.g. ERASING state), implement procedure SET_STATE (BASIC, 1s) and after its successful end, go to the next stage below. If the procedure SET_STATE (BASIC, 1s) fails, consider the procedure LOAD_WRITE_PAGE failed.
11) Increment the write attempts' counter. If it exceeds a specified value, the procedure LOAD_WRITE_PAGE fails. Otherwise, go to the stage 5 above.
12) Increment the attempts' counter. If it exceeds a specified value, the procedure LOAD_WRITE_PAGE fails. Otherwise, go to the stage 7 above.

4. Disclaimer

The content of this document is believed to be correct and accurate. However, the content of this document is furnished "as is" for informational use only and no representation, nor warranty is provided by Melexis about its accuracy, nor about the results of its implementation. Melexis assumes no responsibility or liability for any errors or inaccuracies that may appear in this document. Customer will follow the practices contained in this document under its sole responsibility. This documentation is in fact provided without warranty, term, or condition of any kind, either implied or expressed, including but not limited to warranties of merchantability, satisfactory quality, non-infringement, and fitness for purpose. Melexis, its employees and agents and its affiliates' and their employees and agents will not be responsible for any loss, however arising, from the use of, or reliance on this document. Notwithstanding the foregoing, contractual obligations expressly undertaken in writing by Melexis prevail over this disclaimer.

This document is subject to change without notice, and should not be construed as a commitment by Melexis. Therefore, before placing orders or prior to designing the product into a system, users or any third party should obtain the latest version of the relevant information.
Users or any third party must determine the suitability of the product described in this document for its application, including the level of reliability required and determine whether it is fit for a particular purpose.

This document as well as the product here described may be subject to export control regulations. Be aware that export might require a prior authorization from competent authorities. The product is not designed, authorized or warranted to be suitable in applications requiring extended temperature range and/or unusual environmental requirements. High reliability applications, such as medical life-support or life-sustaining equipment or avionics application are specifically excluded by Melexis. The product may not be used for the following applications subject to export control regulations: the development, production, processing, operation, maintenance, storage, recognition or proliferation of:
1. chemical, biological or nuclear weapons, or for the development, production, maintenance or storage of missiles for such weapons;
2. civil firearms, including spare parts or ammunition for such arms;
3. defense related products, or other material for military use or for law enforcement;
4. any applications that, alone or in combination with other goods, substances or organisms could cause serious harm to persons or goods and that can be used as a means of violence in an armed conflict or any similar violent situation.

No license nor any other right or interest is granted to any of Melexis' or third party's intellectual property rights.

If this document is marked "restricted" or with similar words, or if in any case the content of this document is to be reasonably understood as being confidential, the recipient of this document shall not communicate, nor disclose to any third party, any part of the document without Melexis' express written consent. The recipient shall take all necessary measures to apply and preserve the confidential character of the document. In particular, the recipient shall (i) hold document in confidence with at least the same degree of care by which it maintains the confidentiality of its own proprietary and confidential information, but no less than reasonable care; (ii) restrict the disclosure of the document solely to its employees for the purpose for which this document was received, on a strictly need to know basis and providing that such persons to whom the document is disclosed are bound by confidentiality terms substantially similar to those in this disclaimer; (iii) use the document only in connection with the purpose for which this document was received, and reproduce document only to the extent necessary for such purposes; (iv) not use the document for commercial purposes or to the detriment of Melexis or its customers. The confidentiality obligations set forth in this disclaimer will have indefinite duration and in any case they will be effective for no less than 10 years from the receipt of this document.

This disclaimer will be governed by and construed in accordance with Belgian law and any disputes relating to this disclaimer will be subject to the exclusive jurisdiction of the courts of Brussels, Belgium.

The invalidity or ineffectiveness of any of the provisions of this disclaimer does not affect the validity or effectiveness of the other provisions.
The previous versions of this document are repealed.

IATF 16949 and ISO 14001 Certified